

Which SQL to Choose in System Design

When designing a system, one of the most critical system design choices among SQL vs NoSQL databases can drastically impact your system's overall performance, scalability, and overall success.

What is SQL Database?

Here are some key features of SQL databases:

- **Tabular Data Model:** SQL databases organize records into tables with rows and columns.
- **Fixed Schema:** SQL databases require a predefined schema, which means that you must define the structure of the records, specifying record types and relationships earlier than adding records to the database. This makes SQL databases suitable for steady information.
- **ACID Compliance:** SQL databases are commonly ACID-compliant, which means they make certain records consistency and integrity via Atomicity, Consistency, Isolation, and Durability.
- **Structured Query Language (SQL):** SQL databases use a standardized query language to control and retrieve facts. SQL is strong and supports complex queries, making it suitable for applications requiring statistical analytics and reporting.
- **Strong Relationships:** SQL databases excel in coping with complex relationships between facts tables.

Common examples of SQL databases are:

- **MySQL:** An open-source relational database this is widely utilized in diverse packages.
- **PostgreSQL:** A powerful open-source relational database known for its extensibility and assist for advanced functions.

Where do we use SQL?

- **E-commerce / Financial Systems / CMS :** Managing structured data, relationships, and ensuring transactional integrity and use SQL.
- **Banking & Inventory Management :** Ensures accuracy when transferring funds or updating inventory.

What is NoSQL Database?

NoSQL databases are designed to handle unstructured or semi-based facts and provide flexibility, scalability, and performance. Here are some key capabilities of NoSQL databases:

- **Flexible Data Model:** NoSQL databases use quite a few statistics models, along with key-cost pairs, document stores, huge-column shops, and graph databases.
- **Schema-less:** NoSQL databases are schema-less, this means that statistics can be inserted with out a predefined schema.
- **BASE (Basically Available, Soft State, Eventually Consistent):** Instead of ACID compliance, NoSQL databases frequently comply with the BASE version. BASE prioritizes excessive

NoSQL databases frequently comply with the BASE version. BASE prioritizes excessive availability and performance over strict consistency.

- **Proprietary Query Language:** NoSQL databases typically have their very own query languages tailored to their particular statistics models. These question languages are often more sincere and better acceptable to the facts structure.

Common examples of NoSQL databases consist of:

- **MongoDB:** A popular document shop that is flexible and scalable.
- **Cassandra:** A wide-column shop recognised for its ability to address huge amounts of information and excessive write throughput.

When do we use NoSQL?

- ***Social Media & Big Data Platforms :*** *Handling unstructured user-generated content at scale. Code submissions on GeeksforGeeks Practice.*
- ***Real-Time Analytics, IoT Applications :*** *High write volume, schema flexibility, and the need for fast access.*

SQL vs. NoSQL - Scalability and Performance

- **Vertical Scaling in SQL :** SQL databases traditionally scale vertically by adding more resources to a single server, but this has limitations.
- **Horizontal Scaling in NoSQL :** NoSQL databases shine in horizontal scaling, distributing data across multiple servers to handle increasing loads seamlessly.
- **Considerations for High Traffic Systems :** The scalability requirements of your system and the anticipated traffic should guide your decision on vertical or horizontal scaling.

SQL vs. NoSQL - Query Language and Transactions

- SQL databases use a standardized language for querying data, making it easier for developers familiar with SQL syntax. NoSQL databases vary in their query languages, with some using traditional SQL and others adopting unique approaches.
- The choice between strong ACID transactions (SQL) and eventual consistency (NoSQL) depends on the importance of data integrity in your application.

SQL vs. NoSQL - Flexibility and Schema Evolution

- Adapting a SQL database to evolving data requirements may involve complex schema changes and potential downtime.
- Dynamic schema evolution is supported by NoSQL databases, enabling developers to adjust to shifting requirements with no difficulty.

When should you choose SQL database over NoSQL database in System Design?

SQL databases are appropriate for unique situations, consisting of:

- **Complex Queries:** If your application requires advanced queries and complex reporting, SQL databases excel on this location because of their structured schema and SQL question language.

- **Data Integrity:** When facts consistency and integrity are paramount, particularly in financial or regulatory applications, SQL databases with ACID compliance are the desired preference.
- **Transactions:** SQL databases are the go-to option for packages that require support for multi-step, ACID-compliant transactions, like e-commerce systems.

When should you choose NoSQL database over SQL database in System Design?

NoSQL databases perform better in certain situations:

- **High Scalability:** If your machine wishes to handle a large amount of records and visitors, NoSQL databases provide horizontal scalability, making them a top desire for net and mobile programs.
- **Flexible Schema:** When your information structure is dynamic and may evolve through the years, NoSQL databases with schema-less designs permit for simpler version.
- **Real-time Analytics:** For real-time analytics and processing of streaming facts, NoSQL databases are frequently the favored option due to their pace and versatility.

Differences between SQL and NoSQL

Below are the important differences between SQL and NoSQL:

Aspect	SQL	NoSQL
Data Model and Schema	Enforces a structured schema with predefined tables and relationships.	Embraces a flexible schema, allowing for dynamic and evolving data structures.
Scalability and Performance	Traditionally scales vertically by adding more resources to a single server.	Excels in horizontal scaling, distributing data across multiple servers to handle increasing loads seamlessly.
Query Language and Transactions	Standardized SQL language for querying data.	Varied query languages, with some using SQL and others adopting unique approaches.
Flexibility and Schema Evolution	Rigid schema may require complex changes with potential downtime.	Dynamic schema evolution allows for adaptation to changing data requirements without

		significant disruption.
Use Cases and Applications	Suitable for complex transactions, strict data integrity, and well-defined relationships.	Ideal for applications demanding high scalability, handling large volumes of unstructured data, and rapid development cycles.
Data Integrity and Relationships	Maintains data integrity through ACID transactions. Relationships are well-defined.	Requires denormalization for efficient relationship handling, and the level of consistency may vary.
Security and Authentication	Well-established security mechanisms, including user roles, access controls, and encryption.	Varies in security features, with the need for implementing access controls to protect sensitive data.
Cost Considerations	May involve licensing fees, and scaling vertically can incur higher infrastructure costs.	Often offers cost-effective solutions with horizontal scaling capabilities for handling increased workloads.
Decision Factors for System Design	Align choice with specific project requirements, considering data structures, scalability needs, and development pace.	Evaluate team expertise in SQL or NoSQL, and consider long-term scalability and adaptability aligned with project growth.